

New York University — Tandon School of Engineering

Department of Technology Management and Innovation

IE-GY 9113B — Systems Integration: From ERP to Agentic AI

Study Note — Lab 1: Diagnose & Map a Process

Module 1: The Integration Foundation | Pre-work assigned Week 2 · Lab session Friday, September 18, 2026 |

For this lab week	
Tutorials (self sourced)	Platform tutorials: BPMN in Miro or Lucidchart; Gemini data-generation;
Pre-work	Assigned in Week 2, due before this session: pick a real organisation, research its real friction and value leakages through the Four Lenses, and draft a ½–1 page research brief. The in-class build assumes you arrive with it.
Recall	Week 1 (control points AND decision points, the two flows, the operating model, the leverage gap) and Week 2 (the Four Lenses in full, functional vs. technical requirements, the legacy multi-system reality, obliterate-then-automate). This lab applies them.
Feeds	Board Memo §3 (Integration Backbone Assessment) + §4 — Milestone 1 (problem diagnosis). Feeds capstone storyboard slide S5 (Backbone diagnosis); milestone gloss: Diagnose.

PART ONE · STUDENT GUIDE

1. Why This Lab, and Why It Comes First

This course makes one core argument, and you have now spent two weeks inside it: an organisation's ERP — or, for a smaller company, its integration backbone of stitched-together SaaS tools — is no longer a competitive advantage. It is a *table stake*. The advantage lives in what you build on top of it, layer by layer, from RPA to intelligent automation to autonomous agents. But as Week 2 drove home, **you cannot automate your way out of a broken process** — and you cannot fix what you have not first understood. Lab 1 is where you build that understanding, starting from the business, not the technology.

The order matters, and we do it deliberately: **business first, data last**. You begin with a real organisation and its real problems, map the process — its **control points** (where the business commits) and its **decision points** (where work is routed, approved, or rejected) — and only *then* generate data to fit. This is the Week 1 principle — data hangs on a process — turned into a procedure, and it is how a real implementation team works: map the business before touching the system.

Across the semester your team builds a single board-ready investment case through four progressive labs. This first lab produces the **diagnosis** — the problem half of “what is the problem, and what is the design to close it?” Everything downstream depends on it:

Lab	Question	Default tool	What you do with it
Lab 1 (today)	Where does it hurt, and why?	Research + Miro / Lucidchart + Airtable / Sheets + Gemini	Diagnose the real problem, map the process (control + decision points), build data to prove it.
Lab 2 (Wk 6)	Can we fix the repeatable part?	Make (make.com)	Automate the stable-rules decision point across ≥ 2 services — then break it.
Lab 3 (Wk 9)	What needs judgment, not rules?	Relevance AI	Build an AI agent for the judgment decision point, then stress-test it to failure.
Lab 4 (Wk 12)	How do we run it all — and why fund it?	Miro / Lucidchart	Orchestration blueprint + 3-phase roadmap (value / cost / risk) + board pitch.

The one rule that makes today matter. Do not look for a single pain point; look for a **chain** — a sequence of connected problems where one causes the next. “Invoices go out late” is a symptom. “Inaccurate stock → overselling → order rejected and reworked by hand → delayed invoicing → cash arrives ~12 days late” is a chain that spans Operations, Customer, and Cash — and that is what a board funds.

On tools — outcome-first, stack-flexible

The tools named here are the **Default stack** — free, no-code: Miro/Lucidchart for mapping, Airtable/Google Sheets for the sandbox, Gemini for synthetic data. You may instead meet the lab’s intent on the partner-supported **AWS path** or a **custom stack** of your own design (faculty approval, platform declared by end of Week 3). The graded standard is identical across all three — business outcomes, governance thinking, and solution-design quality, never platform choice. This note describes the Default; substitute your equivalents as you go.

2. How This Lab Works — Before Class vs. In Class

Lab 1 is a **two-part deliverable**. The discovery work happens *before* the session — assigned in Week 2 - so class time is spent building, not staring at a blank page, and so the example patterns we hand out in class sharpen your thinking rather than replace it.

You can start before class — and you should

This lab is built so a motivated team can get a real head start. Everything in Section 3 (pick a company, research the pain, draft your hypotheses) is pre-work you do before class — and you may go further: sketch your process map, mark its control and decision points, and draft your friction chain at home. One sequence rule only: **write your own chain before you open the pattern library in Section 6** — it is a mirror to check your thinking against, not a menu to pick from. Consult it after you have a draft of your own.

Class is for finishing and pressure-testing, not starting from zero. You will refine your map, draw the curveball that tests it, generate the synthetic data, and present.

Arriving with only the pre-work done is completely fine — the session is paced so you can catch up on the mapping in the first block. Arriving with a full draft is better still: you will spend class sharpening a real diagnosis and helping others sharpen theirs.

The shape of the class (2.5 hours)

Time block	What happens
First ~55 min	Refine your map. Finish or tighten the process map — control points (★), decision points (◆), the rejection/rework path (↔), and your friction chain. Catch up here if you arrived with only the pre-work; sharpen here if you arrived with a draft.
~25 min	The curveball. Each team draws a random condition and revises its diagnosis to absorb it.
~25 min	Generate the data. Build the synthetic sandbox (Gemini) to fit your mapped process, including the rejection path — and verify the flaws survived.
Last 50 min	Present & share. Each team gives a 6-minute show-and-tell, then 3 minutes of live feedback on your map (≈ 9 min per team, 4 teams), and the class closes with a 4-minute synthesis. Do not narrate your build — show four things: the map, the friction chain and its cost, what the curveball changed, and your two decision-point targets.

Times are approximate and the front blocks flex; the 50-minute share-out at the end is fixed. Sized for four teams at ~9 minutes each (6 present + 3 feedback) plus a 4-minute close.

Phase	When	What you do	Deliverable
Pre-work	Assigned Wk 2, due before Wk 3	Pick a real organisation; research its real pain and value leakages; draft your own hypotheses — and a first read on its requirements.	A ½–1 page research brief, submitted by email before the lab.
In-lab	Wk 3 session	Map the process, control points, and decision points; trace the rejection/rework path; draw a curveball; generate the data; write the diagnosis.	Sandbox + annotated process map + one-page diagnosis + seeded-flaws key.

The gate — read this. The pre-work research brief is **required and checked before the lab**. Arrive without it and you have no hypotheses to test, the example patterns become a menu you copy rather than a mirror you check against, and you fall behind. The brief need not be polished — it needs to be *yours*, and real.

3. Before the Lab — Pick, Research, Hypothesize

Three steps, done as a team in the days before the Week 3 session. Budget ~1.5–2 hours. This is the *consulting* half of the lab: you walk into an organisation you don't fully understand and form a point of view.

3.1 Step 0 — Confirm Your Company and Reality Brief

Work with **a real organisation** — the company your team confirmed for instructor approval. It can be one a teammate knows from the inside (anonymised) or a company you research from the outside; in every case all sandbox data is *synthetic*. **Submit by email** a one-paragraph reality brief stating four things:

1. **Company & archetype** — industry, rough size, and which of the four archetypes it resembles (Fortune-500 CPG / mid-market manufacturer / Series B startup / small services firm).
2. **Integration stack** — what systems it runs on (e.g., NetSuite; or Stripe + Shopify + QuickBooks). Note any older/legacy tool still in use — Week 2's multi-system reality is real, and it will matter later in this lab.
3. **The business model, in one line** — how it makes money. You can only target the right pain once you understand this.
4. **Your starting suspicion** — the one thread (Customer / Operations & Supply Chain / Cash) you suspect hurts most, and why.

3.2 Step 1 — Research the Real Pain Points and Value Leakages

Investigate using **public sources**: annual reports and 10-Ks, investor-day decks, strategy updates, earnings-call transcripts, analyst notes, and press or Glassdoor for operational pain. You are hunting for **value leakage** — where money, time, or quality escapes the business. If data is not available, make substantiated assumptions. Use Madison’s Four Lenses (taught in full in Week 2) to structure what you look for; name the lens as you find each friction:

Lens	You’re looking for...	Example signal
Frustration	Where people (staff or customers) are angry, blocked, or re-doing work.	“Reps spend hours reconciling orders by hand.”
Time	Where the process is slow or waits pile up.	“Financial close takes 20 days.”
Cost	Where money leaks — rework, write-offs, overtime, penalties.	“Rejected orders are reworked manually, at a cost.”
Quality	Where errors, defects, or bad data occur.	“6% of orders ship wrong.”

Source: Dan Madison, *Process Mapping, Process Improvement and Process Management* (Paton Professional, 2005) — the Four Lenses are his framework for diagnosing a broken process.

Tip

Public companies broadcast their pain in the “Risk Factors” and “MD&A” sections of the 10-K, and in what management promises to fix on earnings calls. Rejection and rework hide in the phrases “manual review,” “exceptions,” and “reprocessing.” Start there.

3.3 Step 2 — Draft Your Own Hypotheses (What You Bring to Class)

Before you see any example patterns, write down **your own** view. This is the deliverable that gates entry to the lab. Include:

1. **Your top 2–3 suspected pain points**, each tagged with its Four-Lens type(s) and the thread it hits.
2. **The critical process(es) behind them** — which end-to-end flow (e.g., Order-to-Cash, Procure-to-Pay, others,..) you think you’ll need to map.

3. **A first read on requirements.** Week 2 showed that mapping a process is how you discover its real requirements. For one or two pain points, note what the business *needs* the system to do (functional) versus how it might have to be built or configured (technical) — and whether either looks mis-specified.
4. **Your first guess at a chain** — how you think these problems connect and cause one another. Rough is fine; you'll refine it in class.

The research brief is a diagnosis, not a description. It is a Four-Lenses read of one real organisation's friction *and* a first pass at its requirements — the raw material you will map, pressure-test, and only then build data for. Bring it, or the lab does not work for your team.

4. In the Lab — Patterns, Mapping, Control & Decision Points

You arrive with your own hypotheses. Now you test and sharpen them, then map.

4.1 Step 3 — Check Your Thinking Against the Pattern Library

Review the **chain-skeleton library** (Section 6) — a set of *causal shapes* that real integration problems tend to take. These are not answers; they are mirrors. Compare them to the chain you drafted in pre-work and choose one path:

1. **Adopt & adapt** — if a skeleton matches what your research found, take its shape and fill it with your company's specifics.
2. **Build your own** — if your research surfaced something the library doesn't cover, build your own chain **to the same standard**: at least 3 causally-linked frictions, spanning at least 2 threads, ending in a board-level consequence.

Do not treat the library as a menu. The skeletons come *after* your research for a reason. If you find yourself abandoning what you discovered to grab a tidy skeleton, stop — that is the exact consulting failure this course warns against: **mapping to your solution instead of to reality**. The library should confirm or challenge your diagnosis, never replace it.

4.2 Step 4 — Map the Process: Control Points, Decision Points, and the Rejection Path

Open Miro or Lucidchart and draw the end-to-end process behind your chain, in **BPMN**. You need the four base shapes, plus three annotations that carry the diagnosis: **control points**, **decision points**, and the **rejection / rework path**.

Element	Canonical meaning	On your map
Rounded rectangle	Start / End	Where the process begins and ends
Rectangle	Task / activity	A step someone or some system does
Horizontal lane	Actor / system	One lane per person or system involved
★ Control point	Where the business commits or data is created	PO issued, serial number generated, invoice posted, credit approved
◆ Decision point (BPMN gateway)	Where the routing choice is made	Approve or reject; route one way or another. NAME each gateway — don't just draw it.
↩ Rejection / rework path	The costly path out of a decision point	Sent back → handled manually → re-enters after significant rework (where delay & error compound)

Control point vs. decision point — hold the distinction. A **control point** (★) is where the business *commits* and a new authoritative fact is created — a PO issued, an invoice posted. A **decision point** (◆) is where work is *routed* — approved, rejected, or sent one way or another. They often sit near each other, but they are not the same: one creates data, the other steers flow. The expensive one is almost always the **rejection path** — work pushed forward, sent back, reworked by hand, and re-entered late. Trace it wherever it exists; it is where your chain's cost usually hides.

Now walk the map and build it out, in order:

1. **Lay out swim-lanes.** One lane per actor/system (Customer, Sales, NetSuite, Warehouse, Finance). Start from a template — Miro: miro.com/templates/bpmn; Lucidchart: [Diagram Use Cases and Tutorials | Lucidchart](#)
2. **Map the real process, not the ideal one.** Draw what actually happens, including the ugly manual workarounds. A suspiciously clean map means you drew your solution, not reality.
3. **Mark every control point with a ★.** PO issued, order confirmed, invoice posted, payment applied. At each, note whether meeting its requirement is standard or a customization (Week 2).

4. **Name every decision point (◆) and diagnose it.** Every BPMN gateway is a routing choice — give it a name (“Credit check: approve/reject,” “Stock available? yes/no”). A drawn-but-unnamed gateway is not yet diagnosed.
5. **Trace the rejection / rework path (↔).** Wherever a decision point can reject or send work back, draw that path: where does rejected work go, who handles it manually, and where does it re-enter? This path is usually the costliest part of the chain — follow it.
6. **Mark the three diagnostic points — two of them ON decision points.**
 - ① The **worst friction** — tag it by Four Lens (Frustration / Time / Cost / Quality). Often this is the rejection/rework path itself.
 - ② A decision point with **stable rules** — the routing choice follows clear, consistent logic. This is your **RPA candidate** and your Lab 2 target.
 - ③ A decision point that needs **judgment** — where two reasonable employees might route it differently. This is your **AI-agent target** and your Lab 3 build.

Seed its inputs now. For this judgment point, name the 3–4 things a person weighs to decide (e.g., which case is more urgent, what a forecast predicts, the cost of delay). You don’t build the full decision data today — but naming the inputs now is what lets you build the Lab 3 agent later. If you can’t name them, this isn’t yet a real judgment point — look again.

You are graded on the thinking, not on flawless software. Your labs use free, no-code tools that sometimes break — and that is fine. A build that runs far enough to show its behaviour and is then broken on purpose, documented honestly, earns full marks; the diagnosis, the design, and the governance argument are the deliverable, not a bug-free demo. Two things follow: (1) each later lab is attemptable **from your sandbox**, not only from the previous lab’s output — a weak Lab 2 does not lock you out of Lab 3. (2) “It broke, here’s our write-up” only counts if you built enough to learn from — productive failure, not no attempt.

7. **Capture requirements as you go (tag “R”).** Beside the relevant steps, jot the functional need and any technical demand, and flag anything mis-specified or whose rationale has been lost over time (Week 2). The map is how you discover requirements.
8. **Draw the chain on the map.** Colored arrows linking the frictions to show how one causes the next; label each link with its thread.

The three diagnostic points are where the whole capstone hinges. Two of your three points sit on **decision points**: the stable-rules one becomes the automation you build in **Lab 2 (Make)**; the judgment one becomes the agent you build and break in **Lab 3 (Relevance AI)**. Choose them deliberately — a mis-chosen decision point makes the next two labs harder. The third point, the worst friction, is usually on the rejection path and anchors your board-level cost.

✓ Test it works

Show the map to a teammate who didn't draw it. If they can find the pain, follow the chain, point to each ★ control point and each named ♦ decision point, and trace where rejected work goes — without you explaining — it works. If not, simplify.

5. In the Lab — the Curveball, the Data, the Diagnosis

5.1 Step 5 — Draw the Curveball Card

Real integration work is never tidy: there is always a reality nobody told you about. This is Week 2's **legacy, multi-system reality** made concrete — and the “decisions you can't take back cheaply” hiding inside a working process. Each team draws one **curveball card** (Section 7): a cross-cutting condition that complicates almost any process. The draw is *random, with instructor veto*. You then **revise your map and diagnosis to absorb it** — including, often, a new rejection path or a decision point that now behaves differently.

1. **Re-map.** Show where the curveball bites — a new lane, a broken hand-off, a decision point that now rejects more often, a reconciliation that fails.
2. **Re-diagnose.** Does it extend your chain? Add a thread? Change which decision point is the RPA candidate vs. the judgment target?
3. **If your curveball is data-relevant** (e.g., “a legacy spreadsheet still runs part of this”), it becomes part of your sandbox next — a small, non-reconciling legacy source.

The curveball is Week 2's reality landing on your own diagnosis. You committed to a view; now the legacy system nobody mentioned, or the vendor that only takes email, pushes back. The point is not to get it right the first time; it is to **update when reality intrudes** — exactly what real transformation work demands.

5.2 Step 6 — Generate the Synthetic Data (Default: Gemini), Fitted to Your Process

Only now do you build the data — to hang on the process you mapped. On the Default stack, use **Gemini** to generate linked synthetic tables, plant your friction chain, and — if your curveball was data-relevant — add a **legacy source that does not reconcile**. Treat any LLM as a fast first draft you **verify and correct**.

Start from your process, not from a template

Your workflow may be Order-to-Cash or Procure-to-Pay — or something quite different (an approval or case-management flow, a service-request or claims process, a scheduling or onboarding flow, and many others). **The tables you need are the ones your process implies, so let Gemini derive them from the map you just built, rather than forcing your process into a fixed schema.**

Give Gemini your process one of three ways (whichever your NYU Gemini supports):

1. **Attach your flowchart.** Export or screenshot your Miro/Lucidchart map and upload the image. Fastest — nothing to rewrite.
2. **Attach or paste a description.** A Word doc or a few lines of text describing the flow.
3. **Fill the five-line template below.** The guaranteed fallback — works in any Gemini and needs no prose. You are transcribing your map into slots, not writing an essay.

“Describe your process” template — paste this, filled in, into Gemini

My process (one end-to-end flow): ____
 Entities that flow through it: ____ (e.g., orders; purchase orders; cases)
 Sub-items attached to each entity: ____ (e.g., order lines; attached documents; sub-records)
 Control points (commit / create): ____ (★ e.g., invoice posted; outcome issued)
 Decision points (approve/reject): ____ (◆ e.g., credit check; case review)
 Rejection path (where sent back, ____
 who reworks it, where it re-enters)

The same structure fits very different workflows. These three are examples only — your process may look like none of them, and that is fine; the point is the shared shape, not the specific case:

O2C (mid-market manufacturer): entities = Orders; sub-items = OrderLines; control points = order confirmed, invoice posted; decision point = credit check (approve/reject); rejection = failed credit → manual review → re-enters late.

P2P (any buyer): entities = Purchase Orders; sub-items = PO lines/receipts; control points = PO issued, goods received, payment released; decision point = three-way-match (pass/fail); rejection = mismatch → manual reconciliation → re-enters.

Approval / case-management (a generic pattern — applications, permits, claims, admissions, and the like): entities = Cases (the applications or requests that flow through); sub-items = the documents or line-items attached to each; control points = case filed, decision recorded, outcome issued; decision point = the review (approve / reject / return-for-more-info); rejection = a returned case → applicant reworks and resubmits → re-enters the queue.

How LLM-generated data fails — read before you prompt

- **It “helpfully” cleans up your flaws.** Ask for an oversold SKU and it may quietly make the numbers consistent. Verify every flaw survived.
- **IDs drift across tables.** Tell it to reuse exact IDs; spot-check the joins.
- **It does more than asked.** Say explicitly: do not add tables, columns, or flaws I didn’t request.

Build data tables (see example below - adapt to your case) (5–7, one end-to-end process, 20–50 rows each)

Table	Minimum fields	Links to
Customers	CustomerID, Name, Segment, PaymentTerms	—
Products	SKU, Description, OnHandQty, ReorderPoint, UnitCost	—
Orders	OrderID, CustomerID, OrderDate, Status (incl. Rejected/Rework)	Customers
OrderLines	LineID, OrderID, SKU, Qty, UnitPrice	Orders, Products
Invoices	InvoiceID, OrderID, Amount, IssueDate, DueDate	Orders
Payments	PaymentID, InvoiceID, Amount, PaidDate	Invoices
Legacy source (if curveball)	e.g., a separate ‘LegacyStock’ sheet with OnHandQty that disagrees with Products	intentionally NOT linked

Note the **Status** field on Orders: include a “Rejected” or “Rework” value so your rejection/rework path is visible *in the data*, not only on the map — this is what makes the decision point diagnosable when you later automate it.

The three prompts (for example table above – adjust to your case as needed)

Prompt A — derive your schema, then generate (paste your filled template or attach your flowchart)

You are helping build a synthetic data sandbox for a university course. Here is my business process: [paste your filled “describe your process” template, OR attach your flowchart image/description]. STEP 1 – From my process, propose the linked tables it needs: one table per entity that flows through the process; a line-item table where an entity has sub-parts; a Status field that can carry the reject/rework path; and fields for the control points and decision points I listed. Show me the proposed table list with columns, and PAUSE – do not generate any rows yet. STEP 2 – After I confirm or correct the schema, generate internally consistent sample data: ~20-50 rows per table, every foreign key matching a real parent row, fake names, 2026 dates, and NO errors yet (clean first). Wait for my go-ahead before Step 2.

Prompt B — plant your chain + the rejection path (on YOUR entities)

Now introduce EXACTLY these flaws into the confirmed dataset, changing only the rows needed and listing every change. Do not fix other data or add flaws I didn't request. [Paste your chosen chain skeleton's flaws, in terms of YOUR entities – e.g., for Order-to-Cash: 1) set 2 SKUs' OnHandQty below committed Qty (oversold); 2) mark ~6 entities Status='Rejected' at your decision point, then re-enter them 8-12 days later as Status='Rework' with a late downstream date; 3) add 3 entities with an ambiguous note requiring judgment.] Keep the flaws connected to the SAME entities so the chain – and the rejection/rework path – is traceable end to end.

Prompt C — legacy mismatch (only if your curveball is data-relevant)

Create a separate 'Legacy' table that covers the same key entities as one of your main tables but with DIFFERENT values for ~10 of them, dated ~2 weeks ago. Do not reconcile it with the main table – the disagreement is the point. Then output a 'Seeded-Flaws Key' table: Flaw #, Type, Table(s)/rows affected, What's wrong, Which next flaw it causes – for the instructor only.

✓ Test it works

Hand-trace ONE flaw end to end: an oversold SKU visible in Products AND in an unfulfillable OrderLine; and a Rejected order that re-enters as Rework with a late invoice. If the rejection path is visible in the Status field and the diagnosis is diagnosable two ways, it's real. If the LLM cleaned it, re-prompt. Whatever your workflow, confirm your sandbox contains the SAME control points (★), decision points (◆), and rejection/rework path (↔) you marked on your map — the data must match the process, not a template.

 Tip

Load into Airtable (build *Link to another record* fields so clicking one record reveals the connected ones) or Google Sheets (match by ID across tabs). AWS/custom teams load into their chosen store. All feed the Lab 2 automation.

5.3 Step 7 — Write the Diagnosis and Present

Turn the map and data into a half-page diagnosis — the connective tissue to your board memo:

1. **The chain, in one sentence** (the connected frictions and the threads they span).
2. **The decision points and rejection path that matter most** — which routing choice is stable-rules (→ Lab 2) and which needs judgment (→ Lab 3), and what the rework path costs.
3. **The board-level consequence, quantified** — a number from your data (“≈12 days of cash tied up in reworked orders”). Order-of-magnitude is fine; it must be a number.
4. **The prize** — “If every link in this chain were addressed, the prize is ____.” A note from Week 2: fixing the whole chain at once is a *Big Bang*; a credible plan phases it — you build that roadmap in Lab 4.

Present, submit (by email) end of day. Sandbox link; process map (★ control points, ◆ named decision points, ⇐ rejection path, the three points, the chain); one-page diagnosis; and the private seeded-flaws key. Each team gives a 6-minute show-and-tell followed by 3 minutes of live feedback: where is the stable-rules decision point (Lab 2)? the judgment one (Lab 3)? and — if it all worked — what bigger thing does it unlock? Come ready to show four things: the annotated map (★ control, ◆ decision, ⇐ rejection), the friction chain and its board-level cost, what the curveball changed, and your two decision-point targets (the Lab 2 automation and the Lab 3 agent).

6. Reference: The Chain-Skeleton Library

Causal *shapes*, not answers. Each is a sequence of linked frictions ending in a board-level consequence; each has a natural decision point where work gets rejected and reworked. Adopt and adapt one — or build your own to the same standard. Threads: **C**=Customer, **O**=Operations/Supply Chain, **\$**=Cash.

Skeleton 1 · Inventory-to-cash [O → C → \$]

Inaccurate stock → overselling → order rejected at fulfilment → manual rework → delayed invoicing → slow cash conversion.

Board-level consequence: Working capital trapped; DSO inflated.

Skeleton 2 · Procurement blind spot [O → \$]

No spend visibility → maverick/duplicate buying → PO rejected/held for approval → late reorder → production delay → expedited freight.

Board-level consequence: Margin erosion; uncontrolled spend.

Skeleton 3 · Multi-system reconciliation [O → C → \$]

Two systems disagree → records fail validation → staff reconcile by hand → errors propagate → customers/finance get wrong numbers.

Board-level consequence: Audit risk; eroded trust in data.

Skeleton 4 · Returns & credit leakage [C → \$]

Manual returns → credit note rejected/held → disputed balances → customers withhold payment → cash & relationship damage.

Board-level consequence: Revenue leakage; churn risk.

Skeleton 5 · Quote-to-onboard drag [C → \$]

Quote/contract routed for manual approval → rejected/reworked → delayed onboarding → delayed first invoice → revenue late.

Board-level consequence: Slow time-to-revenue; forecast unreliability.

Skeleton 6 · Service-to-churn [C → \$]

Fragmented customer data → tickets misrouted/reopened → unresolved issues → dissatisfaction → churn → lost recurring revenue.

Board-level consequence: Retention loss; LTV decline.

7. Reference: The Curveball Deck

Cross-cutting reality conditions — Week 2's legacy, multi-system reality in playable form. Drawn randomly (instructor may veto a true misfit). Cards marked  **data** become part of your sandbox (a non-reconciling legacy source); others reshape the map / diagnosis only.

Card 1 · The hidden legacy system **data**

A spreadsheet (or old tool) nobody mentioned still runs part of this process, and its numbers disagree with the main system.

Card 2 · The acquisition's parallel data **data**

A recently acquired unit kept its own order / customer system; the two have never been merged.

Card 3 · Nightly batch, not real time **data**

A key data source only updates overnight, so decisions are made on yesterday's numbers.

Card 4 · The undocumented manual reconciliation **data**

One person fixes a recurring mismatch by hand every week — and they're about to leave.

Card 5 · The vendor that only takes email / FTP

A critical partner can't do APIs; data moves by emailed file or FTP drop, with lag and errors.

Card 6 · The compliance hold

A new audit / regulatory requirement means a decision point now needs sign-off it didn't before, adding a rejection route.

Card 7 · The seasonal spike

Volume 5x's for one period a year and the process that works at baseline collapses under load.

Card 8 · The duplicate-customer mess **data**

The same customers exist under multiple IDs across systems, so totals never quite add up.

Card 9 · The offline edge case

Part of the process happens offline (paper, phone, a remote site) and only gets keyed in later.

Card 10 · The well-meaning workaround

A team built a shadow process to cope with the system's limits; it "works" but hides the real problem.

Where This Lab Sits — and What It Hands Forward

Lab 1 turns two weeks of concepts into a diagnosis you built by hand. It **recalls** from Week 1 the control points AND decision points, the two flows, the operating model, and the leverage gap; and from Week 2 the Four Lenses, functional-vs-technical requirements, the legacy multi-system reality, and “obliterate before you automate.” It **applies** them to a real organisation: a friction chain, an annotated process map, and a synthetic sandbox. Concretely, it is the six-step “read a backbone” method from Week 1 — trace the flows, mark control and decision points, classify the operating model, run the Four Lenses, listen for warning signs, name the leverage gap — now run by your team on a company of your own.

It **hands forward** three things. To **Lab 2**: the stable-rules decision point becomes the automation you build in Make. To **Lab 3**: the judgment decision point becomes the agent you build and break in Relevance AI. To your **board memo**: this diagnosis is **§3 (Integration Backbone Assessment) and feeds §4 (Automation Journey)** — it is **Milestone 1, the problem diagnosis**. The problem you name today is the one you will still be arguing, at a higher altitude, in December. In the capstone storyboard you receive in Week 1, this diagnosis is slide S5 (Backbone diagnosis) and sits in the first of the course's three milestone phases — Diagnose → Design → Implement. One practical note for the judgment hand-off: the Lab 3 agent needs its decision represented as data, not just named on the map. You seed the inputs today (above); you build the small conflict/decision table itself in the Week 7–8 homework that ramps into Lab 3 — after Lab 2, once you have seen your automation run out of rules on exactly these cases.

Your Study Questions

Bring rough answers; they are designed to be argued, not recited.

1. **Where does your company leak value — and through which of the Four Lenses?** Name the chain, not a single pain, and show how one friction causes the next across at least two threads.
2. **Distinguish a control point from a decision point in your process.** Where does rejected work get sent back, who handles it manually, and what does that rework cost?
3. **Of your decision points, which follows stable rules and which needs judgment?** Say why the first is a safe RPA target (Lab 2) and the second is not (Lab 3).

4. **If every link in your chain were fixed, what is the board-level prize — and why can't you claim it in one move?** Quantify from your data, then say why a phased rollout beats a Big Bang.

Running Glossary

Friction chain. A sequence of causally-linked frictions — one problem causing the next — spanning at least two threads and ending in a board-level consequence. The unit of diagnosis in this course.

Control point. A moment where the business commits and a new authoritative fact is created — a PO issued, a serial number generated, credit approved, an invoice posted. Where governance and automation attach. (Introduced Week 1.)

Decision point. A moment where the routing choice is made — approve or reject, route one way or another; a named BPMN gateway. Distinct from a control point: it steers flow rather than creating data. Two of Lab 1's three diagnostic points sit here. (Introduced Week 1.)

Rejection / rework path. The costly route out of a decision point: work pushed forward is sent back, handled manually to bring it into line, and re-enters the flow only after significant rework — where delay and error compound.

Chain skeleton. A causal shape a friction chain commonly takes (Section 6), offered as a mirror to test your own diagnosis against — never a menu to copy.

Curveball. A cross-cutting reality condition (Section 7) drawn at random in the lab, representing the legacy / multi-system surprises real implementers meet. Data-marked cards enter the sandbox as a non-reconciling source.

Requirement (functional / technical). What a step needs the system to do (functional) versus how it must be built or configured (technical). Introduced Week 2; surfaced on the map here.

The Four Lenses. Frustration, Time, Cost, Quality — four angles for finding where a process leaks value (Madison, 2005). Taught Week 2.

Seeded-flaws key. The private record a team submits listing every planted flaw, the rows it affects, and the link to the next — how the instructor grades the diagnosis.

Appendix – Lab 1 Agenda

Time	Segment	What you do
0:00–0:05	Frame	Confirm each team's status (pre-work only, or map already drafted).
0:05–1:00	Refine / build the map	Finish or tighten the map — ★ control, ◆ named decision points, ↔ rejection path, friction chain.
1:00–1:25	Curveball	Each team draws a card (veto a true misfit); revise map/diagnosis; data-cards flag the legacy step.
1:25–1:40	Data + diagnosis	LLM-generate fitted data incl. the Rejected→Rework path; verify flaws; finalise the diagnosis and a 6-minute show.
1:40–2:26	Presentations + live feedback	4 teams × ~9 min: 6-min show-and-tell, then 3 min of your diagnostic response on that team's map while it is on screen — ask the three checkpoint questions there, not later. Focus on the four things;
2:26–2:30	Closing synthesis	Name the 2–3 strongest chains and the one trap common across the four; confirm EOD submission (map, diagnosis, sandbox link, seeded-flaws key).